



400-650-7088

***Tong*LINK/Q-CN** V10.0

Java 客户端开发手册

版本: 10041A01

2025 年 9 月

声明

版权

Copyright ©2025北京东方通科技股份有限公司或其关联公司（以下简称“东方通”）。保留所有权利。

版权声明

- 本文档的所有权和知识产权归属于东方通所有。
- 除非另有明确约定，东方通不授予任何明示或暗示的许可或权利，使用者不得以任何方式将本文档中的任何内容用于商业目的。
- 东方通对本文档享有版权。本文档的所有内容，包括但不限于文字、图像、图表、图标、示意图、屏幕截图等，均受版权法和国际版权条约的保护。
- 未经东方通明确授权，任何人不得对本文档以任何形式复制、修改、传播、分发、展示或进行衍生创作。

商标声明

- **东方通**、**TongTech** 标识以及其他相关东方通图形、徽标、服务名称和商标（以下统称为“东方通商标”）是东方通或其关联公司的注册商标或商标。
- 未经东方通明确授权，任何人不得以任何方式使用、复制或展示东方通商标。此外，未经东方通事先书面同意，不得将东方通商标与其他商标、标识或徽标进行混淆、链接或结合使用。
- 除非另有明确约定，本商标声明不授予任何明示或暗示的许可或权利，对东方通商标的使用须获得东方通的明确授权。
- 本文档提及的所有其他商标、标识、徽标或产品名称均为其各自所有者的财产，并可能受其各自的商标法保护。

免责声明

请在使用东方通产品之前仔细阅读本免责声明，并根据自身情况判断是否继续使用。如有任何问题，请联系我们的客户支持团队。

- 本产品的使用是基于用户自己的判断和风险评估。本文档仅作为使用指导，不对使用本产品所产生的结果做任何明示或暗示的担保。
- 东方通不对用户未按照本文档中的指导正确使用东方通产品而导致的任何损失或损害承担责任；
- 本文档可能会包含第三方提供的内容、链接或资源，这些内容由第三方自行负责。东方通对于这些内容的准确性、完整性、合法性或可靠性不承担责任。用户在使用这些内容时应自行判断和承担风险。
- 由于产品版本升级或其他原因，本文档内容会不定期更新。东方通保留在不事先通知用户的情况下随时对文档进行修改、更新或中止的权利。

如需获取有关东方通产品的许可或使用权，请联系东方通或授权代理商。任何违反本声明的行为将受到适用法律的追究。

版本变更说明

本手册的更新是累积的。因此，最新的手册版本包含对以前版本所做的所有更改。

本手册版本（10041A01）适用于TongLINK/Q-CN V10.0.4.1。

文档版本	适用产品版本	更新内容	更新日期
10041A01	V10.0.4.1	更新如下内容： 更新版权声明。	2025/9/23
10040A01	V10.0.4.0	更新如下内容： - 第4章 Client：删除“集群级故障转移”相关内容； - 第10章 国密：更新需添加的pom.xml配置内容； 10.2 国密TLS：更新客户端使用示例。	2025/5/23
10030A01	V10.0.3.0	新增如下内容： 10.2 国密TLS：新增章节； - 第11章 消息过滤：新增章节。	2024/11/30
10020A01	V10.0.2.0	无	2024/5/31
10010A01	V10.0.1.0	新增如下内容： 2.3 JDK：新增章节； 9.1 JWT认证：新增章节； 10.1 端到端加密：使用示例中新增参数。 更新如下内容： - 第1章 简介：修改Java客户端版本为3.0.0； - 第9章 认证：删除“TLS认证”、“Athenz”和“Oauth2”章节； 10.1 端到端加密：更新“国密”相关描述。	2023/12/29
10001A01	V10.0.0.1	新增如下内容： 10.1 端到端加密：新增jdk限制描述；使用示例中新增代码段。	2023/10/16
10000A01	V10.0.0.0	TongLINK/Q-CN V10.0第一次正式发布。	2023/7/14

前言

本文档是TongLINK/Q-CN V10.0产品的用户使用手册之一，介绍使用Java客户端生产消费消息的相关概念、配置、参数及示例。

阅读前注意事项

通过阅读本文档，您确认并同意自行承担因未具备必要专业背景 and 知识而导致的任何风险或后果。在使用本文档中提供的信息和指南时，请始终谨慎，并在必要时寻求专业人士建议和指导。

• 适用对象

本文档主要适用于使用本产品的系统管理员阅读，部分内容同样适用于基于本产品进行应用开发或应用部署的人员阅读。

• 专业背景

本文内容可能涉及到操作系统、服务器硬件、网络等相关领域的知识。请确保您具备相关背景 and 知识，以便更好地理解 and 应用本手册的内容。

• 技能要求

为了能够充分理解 and 应用本文档的内容，建议您具备如下技能：

1. 掌握Linux系统基本操作；
2. 掌握K8s的搭建和使用；
3. 熟悉Java开发语言。

• 术语和概念

本文档可能使用一些专业术语和概念。请确保您熟悉这些术语和概念，或者有能力查阅相关资料以便进一步理解。

• 实践经验

为了最大程度地受益于本文档，建议您具备一定实践经验。这将帮助您更好地应用文档中的操作指南 and 建议。

请注意，本文档不适用于没有相关专业背景 and 知识的用户。如果您对本文档的内容 or 所需背景有任何疑问，请在使用之前咨询相关专业人士 or 寻求额外的支持。

用户使用手册集

TongLINK/Q-CN V10.0为您提供的用户使用手册集包含的文档有：

编号	手册集文档	说明
1	000_TongLINK/Q-CN_V10.0用户手册导读	提供手册集的目录及手册版本说明。
2	001_TongLINK/Q-CN_V10.0快速使用手册	提供产品介绍，在K8s和虚拟机上安装部署产品的基本步骤及验证方式。

编号	手册集文档	说明
3	002_TongLINK/Q-CN_V10.0配置参数手册	提供配置文件及参数的详细说明。
4	003_TongLINK/Q-CN_V10.0监控管理手册	提供在K8s和虚拟机上安装部署监控管理组件的基本步骤及验证方式。
5	004_TongLINK/Q-CN_V10.0管理控制台使用手册	详细介绍管理控制台的集群管理、集群监控、告警管理和用户管理相关功能的使用说明。
6	005_TongLINK/Q-CN_V10.0命令行使用手册	详细提供产品中主要命令行的作用、相关参数及使用示例。
7	006_TongLINK/Q-CN_V10.0Java客户端开发手册	介绍使用Java客户端生产消费消息的相关概念、配置、参数及示例。
8	007_TongLINK/Q-CN_V10.0Kafka协议部署手册	提供在K8s和虚拟机上安装部署Kafka协议处理程序的基本步骤及验证方式。
9	008_TongLINK/Q-CN_V10.0AMQP协议部署手册	提供在K8s和虚拟机上安装部署AMQP协议处理程序的基本步骤及验证方式。
10	009_TongLINK/Q-CN_V10.0JMS客户端开发手册	介绍如何将适配ActiveMQ的JMS客户端迁移为适配TongLINK/Q-CN的JMS客户端。
11	010_TongLINK/Q-CN_V10.0MQTT协议部署手册	提供在K8s和虚拟机上安装部署MQTT协议处理程序的基本步骤及验证方式。
12	011_TongLINK/Q-CN_V10.0服务端进阶配置指南	提供在K8s和虚拟机上开启TongLINK/Q-CN的国密通信、消息轨迹、消息过滤等高级功能特性的方法。
13	012_TongLINK/Q-CN_V10.0灾备及故障转移手册	介绍使用灾备及故障转移的相关概念、配置、参数及示例。

技术支持

东方通产品将为您提供全方位的技术支持，您可以通过以下方式获得技术支持：

- 网址：www.tongtech.com
- 电话：400-650-7088
- 邮箱：support@tongtech.com

您在取得技术支持时，请提供如下信息：

- 您的姓名

- 您的公司信息
- 您的联系方式
- 操作系统及其版本
- 产品版本号
- 日志等错误的详细信息

目录

声明	1
版本变更说明	2
前言	3
1. 简介	8
2. 安装	9
2.1. Maven	9
2.2. Gradle	9
2.3. JDK	10
3. 连接配置	11
4. Client	12
4.1. 客户端内存分配器配置	14
5. Producer	16
5.1. 配置生产者	17
5.2. 消息路由	19
5.3. 异步发送	19
5.4. 配置消息	20
5.5. 启用分块	20
5.6. 拦截消息	20
6. Consumer	22
6.1. 配置消费者	23
6.2. 异步接收	28
6.3. 批量接收	29
6.4. 配置分块	30
6.5. nack消息重传	30
6.6. ack超时重传	31
6.7. 多主题订阅	32
6.8. 订阅类型	35
6.8.1. Exclusive	36
6.8.2. Failover	36
6.8.3. Shared	37
6.8.4. Key_shared	38
6.9. 消息拦截器	40
7. Reader	43
7.1. 配置Reader	43
7.2. Sticky key范围Reader	45
8. Schema	46

8.1. AvroBaseStructSchema示例	46
9. 认证	49
9.1. JWT认证	49
10. 国密	50
10.1. 端到端加密	51
10.2. 国密TLS	54
11. 消息过滤	55
11.1. 标签过滤	55
11.1.1. 指定单个tag	56
11.1.2. 指定多个tag	56
11.2. SQL过滤	58
11.2.1. 生产者示例	58
11.2.2. 消费者示例	58
11.2.3. SQL过滤语法规则	60
附录 A: 术语说明	62

1. 简介

您可以使用TongLINK/Q-CN Java客户端创建消息的producer、consumer、readers来生产和消费消息。当前Java客户端版本是**3.0.0**。

Java客户端的producer、consumer、readers中的所有方法都是线程安全的。

客户端的Javadoc按包分为两个域，如下表所示。

包名称	描述	Maven工程
org.apache.pulsar.client.api	生产者和消费者API	org.apache.pulsar:pulsar-client:3.0.0
org.apache.pulsar.client.admin	Java admin API	org.apache.pulsar:pulsar-client-admin:3.0.0
org.apache.pulsar.client.all	包含pulsar-client和pulsar-client-admin包。同时使用了client API和admin API的应用程序可以直接引入此依赖，以减少分别引用所带来的冲突。	org.apache.pulsar:pulsar-client-all:3.0.0

本文档仅关注用于生产和消费主题消息的客户端API。

2. 安装

最新版本的Java客户端库可通过([maven中心仓库](#))引入获得。要使用最新版本，请将客户端依赖添加到构建的配置中。

注意：

- 若不想手动管理依赖冲突，可使用pulsar-client和pulsar-client-admin。pulsar-client和pulsar-client-admin通过([maven-shade-plugin](#))实现隐式依赖，避免底层依赖包（如Netty）的冲突。
- 若需要手动管理依赖项，可使用pulsar-client-original和pulsar-client-admin-original。pulsar-client-original和pulsar-client-admin-original不会隐藏依赖性。

2.1. Maven

若使用Maven，请在pom.xml文件中添加以下内容：

- 在<properties>小节中添加如下内容：

```
<pulsar.version>4.0.3</pulsar.version>
```

- 在<dependencies>小节中添加如下内容：

```
<dependency>

    <groupId>org.apache.pulsar</groupId>

    <artifactId>pulsar-client</artifactId>

    <version>${pulsar.version}</version>

</dependency>
```

2.2. Gradle

若使用Gradle，请在build.gradle文件中添加以下内容：

```
def pulsarVersion = '3.0.0'

dependencies {

    compile group: 'org.apache.pulsar', name: 'pulsar-client', version: pulsarVersion

}
```

2.3. JDK

如果要使用端到端加密，请使用**JDK 1.8 8u161**之后到JDK11之间的版本。不使用端到端加密，可使用JDK1.8以上的版本。

3. 连接配置

要使用客户端库连接到TongLINK/Q-CN，需要指定一个URL，默认端口为6650。示例如下：

pulsar://localhost:6650

若包含多个代理，则URL配置如下：

pulsar://localhost:6650,localhost:6651,localhost:6652

使用域名配置统一的集群地址如下：

pulsar://pulsar.us-west.example.com:6650

如果使用TLS身份验证，则URL如下：

pulsar+ssl://pulsar.us-west.example.com:6651

4. Client

可以仅使用TongLINK/Q-CN集群的一个URL来实例化PulsarClient对象，如下所示：

```
PulsarClient client = PulsarClient.builder()

    .serviceUrl("pulsar://localhost:6650")

    .build();
```

若存在多个代理，也可以使用该方式启动一个PulsarClient，如下所示：

```
PulsarClient client = PulsarClient.builder()

    .serviceUrl("pulsar://localhost:6650,localhost:6651,localhost:6652")

    .build();
```

如果创建客户端，则可以使用loadConf配置。在loadConf中可使用以下参数配置。

参数名	类型	描述	默认
serviceUrl	String	TongLINK/Q-CN集群地址。	-
authPluginClassName	String	身份验证插件的名称。	-
authParams	String	身份验证插件的参数。 示例： key1:val1,key2:val2	-
operationTimeoutMs	long	操作超时时间，单位秒。 在此时间段内生产者创建、订阅和取消订阅操作会一直重试，若超过这个时间还没成功将会被标记失败。	30
statsIntervalSeconds	long	每次统计信息之间的间隔，单位秒。 Stats被激活，statsInterval为正值； statsIntervalSeconds至少设置为1秒。	60
numIoThreads	int	用于处理与broker连接的线程数。	1

参数名	类型	描述	默认
numListenerThreads	int	用于处理消息监听器的线程数。 监听器线程池在所有使用“监听器”模型获取消息的消费者和读者之间共享。对于一个给定的消费者，监听器始终从同一线程调用，以确保消息排序。 如果需要多个线程处理单个主题，则要创建一个共享订阅和这个订阅的多个消费者，但无法确保消息有序。	1
useTcpNoDelay	boolean	是否在TCP连接上使用无延迟标志来禁用Nagle算法。	True
enableTls	boolean	是否在连接上使用TLS加密。 注意：此参数已弃用。如果要启用TLS，请改用pulsar+ssl://in serviceUrl。	False
tlsTrustCertsFilePath	String	受信任的TLS证书文件的路径。	false
tlsAllowInsecureConnection	boolean	客户端是否接受来自broker的不受信任的TLS证书。	False
tlsHostnameVerificationEnable	boolean	是否启用TLS主机名验证。	False
concurrentLookupRequest	int	每个broker连接上允许发送的并发查找请求数，防止broker过载。	5000
maxLookupRequest	int	每个broker连接上允许的最大查找请求数，防止broker过载。	50000
maxNumberOfRejectedRequestPerConnection	int	当前连接关闭后，客户端创建新连接连接不同broker后一定时间范围内（60秒）broker拒绝请求的最大次数。	50
keepAliveIntervalSeconds	int	每个客户端与broker的连接保持活跃的时间，单位秒。	30
connectionTimeoutMs	int	客户端与broker建立连接的超时时间，单位毫秒。	10000
requestTimeoutMs	int	完成请求的最长持续时间，单位毫秒。	60000

参数名	类型	描述	默认
defaultBackoffIntervalNanos	int	默认重试间隔时间，单位纳秒。	TimeUnit.MILLISECONDS.toNanos(100);
maxBackoffIntervalNanos	长	重试间隔的最长持续时间，单位纳秒。	TimeUnit.SECONDS.toNanos(30)
socks5ProxyAddress	SocketAddress	SOCKS5代理地址。	-
socks5ProxyUsername	String	SOCKS5代理用户名。	-
socks5ProxyPassword	String	SOCKS5代理密码。	-

除了客户端级配置之外，生产者和消费者还可以应用特定的配置，如以下章节所述。

4.1. 客户端内存分配器配置

可通过Java properties设置客户端内存分配器配置。

属性	类型	描述	默认	可用值
pulsar allocator.pooled	String	- 如果设置为true，则客户端使用直接内存池； - 如果设置为false，则客户端使用没有池的堆内存。	True	- True - False
pulsar allocator.exit_on_oom	String	OOM时是否退出JVM	False	- True - False
pulsar allocator.leak_detection	String	Pulsar bytebuf分配器的泄漏检测策略。 Disabled：无泄漏检测且无开销。 Simple：检测1%的已分配缓冲区来跟踪泄漏。 Advanced：检测1%的已分配缓冲区来跟踪泄漏，报告缓冲区堆栈跟踪的位置。 Paranoid：检测100%的已分配缓冲区来跟踪泄漏，报告缓冲区堆栈跟踪的位置并引入一个显著的开销。	Disabled	- Disabled - Simple - Advanced - Paranoid

属性	类型	描述	默认	可用值
pulsar allocator.out_of_memory_policy	String	当OOM发生时，客户端抛出异常或者回退到堆中。	FallbackToHeap	- ThrowException - FallbackToHeap

示例:

```
-Dpulsar.allocator.pooled=true

-Dpulsar.allocator.exit_on_oom=false

-Dpulsar.allocator.leak_detection=Disabled

-Dpulsar.allocator.out_of_memory_policy=ThrowException
```


5. Producer

在TongLINK/Q-CN中，生产者将消息写入主题。一旦实例化一个PulsarClient对象，就可以为特定的主题创建一个生产者。

```
Producer<byte[]> producer = client.newProducer()

    .topic("my-topic")

    .create();

//向指定的broker和主题发送消息

producer.send("My message".getBytes());
```

默认情况下，生产者生成由字节数组组成的消息。用户可通过指定一个消息schema来生成不同的类型。

```
Producer<String> stringProducer = client.newProducer(Schema.STRING)

    .topic("my-topic")

    .create();

stringProducer.send("My message");
```

确保在不需要时关闭生产者、消费者和客户端。

```
producer.close();

consumer.close();

client.close();
```

关闭操作也可以是异步的：

```
producer.closeAsync()

    .thenRun(() -> System.out.println("Producer closed"))

    .exceptionally((ex) -> {

        System.err.println("Failed to close producer: " + ex);

        return _null;
```

```
});
```

5.1. 配置生产者

如果Producer如上述示例只指定主题名来实例化对象，则使用生产者的默认配置。

如果创建生产者，则可以使用loadConf配置。在loadConf可使用以下参数配置：

属性名	类型	描述	默认值
topicName	String	主题名称	-
producerName	String	生产者名称	-
sendTimeoutMs	long	消息发送超时时间，单位ms。 如果在sendTimeout过期前，消息未被服务器确认，则会发生错误。	30000
blockIfQueueFull	boolean	- 如果设置为true，当传出的消息队列已满时，生产者的Send和SendAsync方法会阻塞，而不是失败并抛出错误。 - 如果设置为false，当传出的消息队列已满时，生产者的Send和SendAsync方法会失败，并出现ProducerQueueFullError异常。 MaxPendingMessages参数决定传出消息队列的大小。	False
maxPendingMessages	int	保存挂起消息的队列大小的最大值。 例如，等待从broker接收确认的消息。 默认情况下，当队列已满时，除非设置BlockIfQueueFull为true，否则所有对Send和SendAsync方法的调用都会失败。	1000
maxPendingMessagesAcrossPartitions	int	各个分区的最大待处理消息数。 如果总数超过配置值，请使用该设置降低每个分区的最大待处理消息数(@link #setMaxPendingMessages(int))。	50000

属性名	类型	描述	默认值
messageRoutingMode	MessageRoutingMode	分区主题上生产者的消息路由逻辑。 只有对消息设置无键时才应用该逻辑。 可选项如下： - pulsar.RoundRobinDistribution: 轮询 - pulsar.UseSinglePartition: 将所有消息发布到一个分区上 - pulsar.CustomPartition: 自定义分区方案	pulsar.RoundRobinDistribution
hashingScheme	HashingScheme	确定发布特定消息的分区的哈希函数（仅适用于分区主题）。 可选项如下： - pulsar.JavastringHash: 相当于Java中的string.hashCode() - pulsar.Murmur3_32Hash: 应用Murmur3的散列函数 - pulsar.BoostHash: 应用C++的Boost库中的散列函数	HashingScheme.JavastringHash
cryptoFailureAction	ProducerCryptoFailureAction	加密失败时，生产者应采取措施。 - FAIL: 如果加密失败，则未加密的消息将无法发送。 - SEND: 如果加密失败，则发送未加密的消息。	ProducerCryptoFailureAction.FAIL
batchingMaxPublishDelayMicros	long	批量发送消息的时间段。	TimeUnit.MILLISECONDS.toMicros(1)
batchingMaxMessages	int	批处理中允许的最大消息数。	1000
batchingEnabled	boolean	启用消息批处理。	True
chunkingEnabled	boolean	启用消息分块。	False

属性名	类型	描述	默认值
compressionType	CompressionType	生产者使用的消息数据压缩类型。 可选项如下： • LZ4 • ZLIB • ZSTD • SNAPPY	-
initialSubscriptionName	String	使用此配置，在创建主题时自动创建一个初始订阅。如果未设置此字段，则不会创建初始订阅。	-

如果不想使用默认配置，可以配置参数。示例如下：

```

Producer<byte[]> producer = client.newProducer()

    .topic("my-topic")

    .batchingMaxPublishDelay(10, TimeUnit.MILLISECONDS)

    .sendTimeout(10, TimeUnit.SECONDS)

    .blockIfQueueFull(true)

    .create();

```

5.2. 消息路由

使用分区主题时，可以在使用生产者发布消息时指定路由模式。

5.3. 异步发送

用户可使用Java客户端异步发送消息。使用异步发送，生产者将消息放入一个阻塞队列并立即返回。然后客户端库在后台将消息发送给broker。如果队列已满（最大大小可配置），生产者在调用API时会被阻塞或立即失败，具体取决于传递给生产者的参数。

示例如下：

```

producer.sendAsync("my-async-message".getBytes()).thenAccept(msgId -> {

    System.out.println("Message with ID " + msgId + " successfully sent");

});

```

从以上示例可以看出，异步发送操作返回一个包装在CompletableFuture中的MessageId。

5.4. 配置消息

除了值之外，还可以在给定消息上设置其他项目：

```
producer.newMessage()

    .key("my-message-key")

    .value("my-async-message".getBytes())

    .property("my-key", "my-value")

    .property("my-other-key", "my-other-value")

    .send();
```

可以使用sendAsync()终止构建器链，并返回获得future。

5.5. 启用分块

消息分块使TongLINK/Q-CN能够通过在生产者端将消息拆分为块并在消费者端聚合分块消息来处理大负载消息。

默认情况下，消息分块功能处于关闭状态。以下是关于如何在创建生产者时启用消息分块的示例。

```
Producer<byte[]> producer = client.newProducer()

    .topic(topic)

    .enableChunking(true)

    .enableBatching(false)

    .create();
```

默认情况下，生产者根据broker配置的最大消息大小（maxMessageSize）对大消息进行分块处理（例如：5MB）。但是，客户端也可以使用生产者配置（chunkMaxMessageSize）来配置最大分块的大小。

注意：要启用分块，需要同时禁用批处理（enableBatching = false）。

5.6. 拦截消息

ProducerInterceptors拦截生产者发布的消息，并可能在消息发布到broker之前对其进行改变。

该接口有三个主要事件：

- eligible检查拦截器是否可以应用于消息。
- beforeSend在生产者将消息发送给broker之前触发。用户可修改此事件中的消息。
- onSendAcknowledgement当消息被broker确认或发送失败时触发。

为了拦截消息，可以在创建生产者时添加一个或多个ProducerInterceptor，如下所示。

```

Producer<byte[]> producer = client.newProducer()

    .topic(topic)

    .intercept(_new_ ProducerInterceptor {

        @Override

        boolean eligible(Message message) {

            return true; // process all messages

        }

        @Override

        Message beforeSend(Producer producer, Message message) {

            //用户自定义的处理逻辑

        }

        @Override

        void onSendAcknowledgement(Producer producer, Message message, MessageId msgId, Throwable
exception) {

            //用户自定义的处理逻辑

        }

    })

    .create();

```

注意：如果使用多个拦截器，它们将按照传递给intercept方法的顺序应用。

6. Consumer

在TongLINK/Q-CN中，消费者订阅主题并处理生产者发布到这些主题的消息。首先通过实例化PulsarClient对象，并将其传递给TongLINK/Q-CN broker的URL来实例化一个新的消费者。

实例化PulsarClient对象后，可以通过指定主题和订阅来创建消费者。

```
Consumer consumer = client.newConsumer()

    .topic("my-topic")

    .subscriptionName("my-subscription")

    .subscribe();
```

该subscribe方法将自动为消费者订阅指定的主题和订阅。让消费者监听主题的一种方法是设置一个while循环。在此示例的循环中，消费者监听消息，打印任何接收到的消息内容，然后确认消息已被处理。如果处理逻辑失败，可以使用否定确认并在稍后重新发送消息。

```
while (true) {

    //等待消息

    Message msg = consumer.receive();

    try {

        //打印消息内容

        System.out.println("Message received: " + new String(msg.getData()));

        //确认消息，以便broker删除该消息

        consumer.acknowledge(msg);

    } catch (Exception e) {

        //消息处理失败，稍后重新发送

        consumer.negativeAcknowledge(msg);

    }

}
```

如果不想阻塞主线程而又要不断监听新的消息，可使用MessageListener。

```
MessageListener myMessageListener = (consumer, msg) -> {

    try {

        System.out.println("Message received: " + new String(msg.getData()));

        consumer.acknowledge(msg);

    } catch (Exception e) {

        consumer.negativeAcknowledge(msg);

    }

}

Consumer consumer = client.newConsumer()

    .topic("my-topic")

    .subscriptionName("my-subscription")

    .messageListener(myMessageListener)

    .subscribe();
```

6.1. 配置消费者

如果同上述示例，仅指定主题和订阅名称来实例化Consumer对象，则Consumer使用默认配置。

创建消费者时，可以使用loadConf配置。在loadConf中可使用以下参数配置。

参数名	类型	描述	默认值
topicNames	Set<String>	主题名称	Sets
topicsPattern	Pattern	主题正则表达式	-
subscription Name	String	订阅名称	-

参数名	类型	描述	默认值
subscriptionType	SubscriptionType	订阅类型。 提供四种订阅类型： • Exclusive • Failover • Shared • Key_Shared	SubscriptionType.Exclusive
receiverQueueSize	int	消费者的接收者队列大小。 例如，在应用程序调用Receive之前，消费者累积的消息数。 高于默认值的值会增加用户吞吐量，但会牺牲更多的内存利用率。	1000
acknowledgmentsGroupTimeMicros	long	对指定时间的消费者确认分组。 默认情况下，消费者向broker发送确认消息的分组时间为100毫秒。 将分组时间设置为0会立即发送确认消息。 更长的分组时间更有效，但失败后消息的重新传递会略有增加。	TimeUnit.MILLISECONDS.toMicros(100)
negativeAckRedeliveryDelayMicros	long	在重新发送处理失败的消息之前等待的时间。 当应用程序使用{@link Consumer#negativeAcknowledge(Message)}时，失败的消息会在固定超时时间后重新发送。	TimeUnit.MINUTES.toMicros(1)
maxTotalReceiverQueueSizeAcrossPartitions	int	各个分区的最大总接收器队列大小。 如果总接收器队列大小超过此值，则此设置会减少单个分区的接收器队列大小。	50000
consumerName	String	消费者名称	-
ackTimeoutMillis	long	未确认消息的超时时间	0

参数名	类型	描述	默认值
tickDuration Millis	long	ack-timeout重新传递的粒度。 使用较高的tickDurationMillis可以减少在设置ack-timeout为较大值（例如，1小时）时跟踪消息的内存开销。	1000
priorityLevel	int	消费者的优先级，broker在共享订阅模式下分发消息时给予其更高的优先级。 broker遵循递减的优先级。例如，0=max-priority, 1, 2,... 在共享订阅模式下，broker首先将消息分发给有许可证的最大优先级消费者。否则，broker会考虑下一个优先级消费者。 示例1: 如果订阅的消费者A为priorityLevel0，消费者B为priorityLevel1，则broker仅向消费者A发送消息，直到用完许可，然后开始向消费者B发送消息。 示例2: 消费者优先级，级别，许可为： C1, 0, 2 C2, 0, 1 C3, 0, 1 C4, 1, 2 C5, 1, 1 broker向消费者发送消息的顺序是：C1, C2, C3, C1, C4, C5, C4。	0

参数名	类型	描述	默认值
cryptoFailureAction	ConsumerCryptoFailureAction	当消费者在收到无法解密的消息时，应采取措施。 • FAIL: 默认值，在加密成功之前，消息都是失败的。 • DISCARD: 默默确认，并且不向应用程序传递消息。 • CONSUME: 将加密消息传递给应用程序。解密消息由应用程序负责。 消息解压失败。 如果消息包含批处理消息，则客户端无法批量检索单个消息。 传递的加密消息包含{@link EncryptionContext}，其中包含加密和压缩信息，应用程序可以使用这些信息解密消耗的消息有效负载。	ConsumerCryptoFailureAction.FAIL
properties	SortedMap<String, String>	消费者的名称或属性。 properties是附加到消费者上的应用程序定义的元数据。 当获取主题统计信息时，将此元数据与消费者统计信息相关联以便于识别。	new TreeMap()
readCompacted	boolean	如果启用readCompacted，消费者将从一个压缩的主题中读取消息，而不是读取一个主题的全部消息积压。 消费者只能看到压缩主题中每个键的最新值，直到达到主题消息中压缩积压的点。除此之外，照常发送消息。 只在持久主题的订阅上启用readCompacted，这些主题具有单个活跃消费者（如灾备灾备(Failover)或独占(Exclusive)订阅）。 尝试在订阅非持久主题或共享订阅时启用它会导致订阅调用抛出PulsarClientException。	False
subscriptionInitialPosition	SubscriptionInitialPosition	第一次订阅主题时，设置订阅的初始位置。	SubscriptionInitialPosition.Latest
patternAutoDiscoveryPeriod	int	使用主题消费者模式时，主题的自动发现周期。 默认值和最小值是1分钟。	1

参数名	类型	描述	默认值
regexSubscriptionMode	RegexSubscriptionMode	使用正则表达式订阅主题时，可以选择特定类型的主题。 - PersistentOnly: 只订阅持久主题。 - NonPersistentOnly: 只订阅非持久主题。 - AllTopics: 订阅持久性和非持久性主题。	RegexSubscriptionMode.PersistentOnly
deadLetterPolicy	DeadLetterPolicy	消费者的死信策略。 默认情况下，某些消息可能会被多次重发，甚至达到永不停止的程度。 通过使用死信机制，消息具有最大的重传次数。当超过最大重传次数时，消息将发送到死信主题并自动确认。 可通过设置deadLetterPolicy来启用死信机制。 示例： <pre>client.newConsumer() .deadLetterPolicy(DeadLetterPolicy.builder().maxRedeliverCount(10).build()) .subscribe();</pre> 默认的死信主题名称为{TopicName}-{Subscription}-DLQ 设置一个自定义的死信主题名称： <pre>client.newConsumer() .deadLetterPolicy(DeadLetterPolicy.builder().maxRedeliverCount(10) .deadLetterTopic("your-topic-name").build()) .subscribe();</pre> 当指定死信策略而不指定ackTimeoutMillis时，可以将ack超时设置为30000毫秒。	-
autoUpdatePartitions	boolean	如果autoUpdatePartitions启用，消费者会自动订阅分区增量。 注意： 仅适用于分区消费者。	True

参数名	类型	描述	默认值
replicateSubscriptionState	boolean	如果replicateSubscriptionState启用，订阅状态将被复制到异地复制集群。	False
negativeAckRedeliveryBackoff	RedeliveryBackoff	自定义消息的接口是negativeAcked策略。可以为消费者指定RedeliveryBackoff。	MultiplierRedeliveryBackoff
ackTimeoutRedeliveryBackoff	RedeliveryBackoff	自定义消息的接口是ackTimeout策略。可以为消费者指定RedeliveryBackoff。	MultiplierRedeliveryBackoff
autoAckOldestChunkedMessageOnQueueFull	boolean	达到maxPendingChunkedMessage的阈值时，是否自动确认挂起的分块消息。如果设置为false，这些消息将由其broker重新发送。	True
maxPendingChunkedMessage	int	保存待处理分块消息的队列的最大大小。当达到阈值时，消费者会丢弃挂起的消息以优化内存利用率。	10
expireTimeOfIncompleteChunkedMessageMillis	long	如果消费者未能在指定时间段内接收到所有块，使不完整块过期的时间间隔。默认值为1分钟。	60000

如果不想使用默认配置，可以配置参数。示例如下：

```
Consumer consumer = client.newConsumer()

    .topic("my-topic")

    .subscriptionName("my-subscription")

    .ackTimeout(10, TimeUnit.SECONDS)

    .subscriptionType(SubscriptionType.Exclusive)

    .subscribe();
```

6.2. 异步接收

receive方法同步接收消息（消费者进程被阻塞，直到消息可用）。还可以使用异步接收，一旦有新消息可用，则立即返回一个CompletableFuture对象。

示例如下：

```
CompletableFuture<Message> asyncMessage = consumer.receiveAsync();
```

异步接收操作返回包装在CompletableFuture中的消息。

6.3. 批量接收

使用batchReceive接收每次调用的多条消息。

示例如下：

```
Messages messages = consumer.batchReceive();

for (Object message : messages) {

    //处理消息

}

consumer.acknowledge(messages)
```

注意：

批量接收策略限制单个批次中消息的数量和字节。可以指定超时以等待足够的消息。如果满足以下任一条件，则批量接收完成：足够的消息数、消息字节数、等待超时。

```
Consumer consumer = client.newConsumer()

.topic("my-topic")

.subscriptionName("my-subscription")

.batchReceivePolicy(BatchReceivePolicy.builder()

.maxNumMessages(100)

.maxNumBytes(1024 * 1024)

.timeout(200, TimeUnit.MILLISECONDS)

.build())
```

```
.subscribe();
```

默认的批量接收策略为：

```
BatchReceivePolicy.builder()

.maxNumMessage(-1)

.maxNumBytes(10 * 1024 * 1024)

.timeout(100, TimeUnit.MILLISECONDS)

.build();
```

6.4. 配置分块

通过配置maxPendingChunkedMessage和autoAckOldestChunkedMessageOnQueueFull参数，可以限制消费者同时维护的最大分块消息数。当达到阈值时，消费者通过静默确认或要求broker稍后重新提交待处理消息来丢弃这些消息。expireTimeOfIncompleteChunkedMessage参数决定消费者未能在指定的时间段内接收到消息的所有块，使不完整块过期的时间间隔。

以下是如何配置消息分块的示例。

```
Consumer<byte[]> consumer = client.newConsumer()

.topic(topic)

.subscriptionName("test")

.autoAckOldestChunkedMessageOnQueueFull(true)

.maxPendingChunkedMessage(100)

.expireTimeOfIncompleteChunkedMessage(10, TimeUnit.MINUTES)

.subscribe();
```

6.5. nack消息重传

RedeliveryBackoff引入消息重传的延迟机制。通过设置消息的重传次数，可以实现不同延迟的重传。

```
Consumer consumer = client.newConsumer()

.topic("my-topic")
```

```
.subscriptionName("my-subscription")

.negativeAckRedeliveryBackoff(MultiplierRedeliveryBackoff.builder()

    .minDelayMs(1000)

    .maxDelayMs(60 * 1000)

    .build())

.subscribe();
```

6.6. ack超时重传

RedeliveryBackoff引入消息的重传机制。可以通过设置重传消息的次数来以不同的延迟重新发送消息。

```
Consumer consumer = client.newConsumer()

    .topic("my-topic")

    .subscriptionName("my-subscription")

    .ackTimeout(10, TimeUnit.SECOND)

    .ackTimeoutRedeliveryBackoff(MultiplierRedeliveryBackoff.builder()

        .minDelayMs(1000)

        .maxDelayMs(60000)

        .multiplier(2)

        .build())

    .subscribe();
```

消息重传行为如下所示：

重传次数	重传延迟
1	10 + 1 秒
2	10 + 2 秒

重传次数	重传延迟
3	10 + 4 秒
4	10 + 8 秒
5	10 + 16 秒
6	10 + 32 秒
7	10 + 60 秒
8	10 + 60 秒

注意：

- negativeAckRedeliveryBackoff不适用于consumer.negativeAcknowledge(MessageId messageId)，因为无法从消息ID获取重传次数。
- 如果消费者崩溃，将触发未确认消息的重传。在这种情况下，RedeliveryBackoff不会生效，消息可能会在backoff的延迟时间之前被重新传递。

6.7. 多主题订阅

除了将消费者订阅到单一的主题外，还可以使用多主题订阅同时订阅多个主题。要使用多主题订阅，需要提供一个正则表达式（regex）或一个List主题。如果通过正则表达式选择主题，所有主题都必须在同一个命名空间中。

示例如下：

```
import org.apache.pulsar.client.api.Consumer;

import org.apache.pulsar.client.api.PulsarClient;

import java.util.Arrays;

import java.util.List;

import java.util.regex.Pattern;

ConsumerBuilder consumerBuilder = pulsarClient.newConsumer()

    .subscriptionName(subscription);

//订阅一个命名空间中的所有主题

Pattern allTopicsInNamespace = Pattern.compile("public/default/.*");
```

```
Consumer allTopicsConsumer = consumerBuilder

    .topicsPattern(allTopicsInNamespace)

    .subscribe();

//在一个命名空间中，根据搜索结果订阅一个主题子集

Pattern someTopicsInNamespace = Pattern.compile("public/default/foo.*");

Consumer allTopicsConsumer = consumerBuilder

    .topicsPattern(someTopicsInNamespace)

    .subscribe();
```

在上面的示例中，消费者订阅了可以匹配主题名称规则的持久性主题。如果想让消费者订阅与主题名称规则匹配的所有持久性和非持久性主题，请将subscriptionTopicsMode设置为RegexSubscriptionMode.AllTopics。

```
Pattern pattern = Pattern.compile("public/default/.*");

pulsarClient.newConsumer()

    .subscriptionName("my-sub")

    .topicsPattern(pattern)

    .subscriptionTopicsMode(RegexSubscriptionMode.AllTopics)

    .subscribe();
```

注意：默认情况下，消费者的subscriptionTopicsMode为PersistentOnly。subscriptionTopicsMode的可用选项为PersistentOnly、NonPersistentOnly和AllTopics。

还可以订阅明确的主题列表（支持跨命名空间）：

```
List<String> topics = Arrays.asList(

    "topic-1",

    "topic-2",

    "topic-3"
```

```
);

Consumer multiTopicConsumer = consumerBuilder

    .topics(topics)

    .subscribe();

//其他方案

Consumer multiTopicConsumer = consumerBuilder

    .topic(

        "topic-1",

        "topic-2",

        "topic-3"

    )

    .subscribe();
```

除同步订阅方法外，还可以使用subscribeAsync方法异步订阅多个主题。示例如下：

```
Pattern allTopicsInNamespace = Pattern.compile("persistent://public/default.*");

consumerBuilder

    .topics(topics)

    .subscribeAsync()

    .thenAccept(_this_::receiveMessageFromConsumer);

private void receiveMessageFromConsumer(Object consumer) {

    ((Consumer)consumer).receiveAsync().thenAccept(message -> {

        //处理收到的消息

        receiveMessageFromConsumer(consumer);

    });
}
```

```
});  
  
}
```

6.8. 订阅类型

TongLINK/Q-CN有多种订阅类型来匹配不同的场景。一个主题可以有多个不同订阅类型的订阅。但是，一次订阅只能有一种订阅类型。

订阅与订阅名称相同；订阅名称一次只能指定一种订阅类型。要更改订阅类型，首先需要使该订阅的所有消费者离线。

不同的订阅类型有不同的消息分发类型。本节介绍订阅类型的差异及其使用方法。

为了更好地描述它们的区别，假设有一个名为“my-topic”的主题，并且生产者已发布10条消息。

```
Producer<String> producer = client.newProducer(Schema.STRING)  
  
    .topic("my-topic")  
  
    .enableBatching(false)  
  
    .create();  
  
// 3条"key-1"消息， 3条"key-2"消息， 2条"key-3"消息， 2条"key-4"消息  
  
producer.newMessage().key("key-1").value("message-1-1").send();  
  
producer.newMessage().key("key-1").value("message-1-2").send();  
  
producer.newMessage().key("key-1").value("message-1-3").send();  
  
producer.newMessage().key("key-2").value("message-2-1").send();  
  
producer.newMessage().key("key-2").value("message-2-2").send();  
  
producer.newMessage().key("key-2").value("message-2-3").send();  
  
producer.newMessage().key("key-3").value("message-3-1").send();  
  
producer.newMessage().key("key-3").value("message-3-2").send();  
  
producer.newMessage().key("key-4").value("message-4-1").send();  
  
producer.newMessage().key("key-4").value("message-4-2").send();
```

6.8.1. Exclusive

创建一个新的消费者并使用Exclusive订阅类型订阅。

```
Consumer consumer = client.newConsumer()

    .topic("my-topic")

    .subscriptionName("my-subscription")

    .subscriptionType(SubscriptionType.Exclusive)

    .subscribe()
```

只允许第一个消费者订阅，其他消费者会收到错误。第一个消费者收到全部10条消息，消费顺序与生产顺序相同。

注意：如果主题是分区主题，则第一个消费者订阅所有分区主题，其他消费者未分配分区并收到错误。

6.8.2. Failover

创建新的消费者并使用Failover订阅类型订阅。

```
Consumer consumer1 = client.newConsumer()

    .topic("my-topic")

    .subscriptionName("my-subscription")

    .subscriptionType(SubscriptionType.Failover)

    .subscribe()

Consumer consumer2 = client.newConsumer()

    .topic("my-topic")

    .subscriptionName("my-subscription")

    .subscriptionType(SubscriptionType.Failover)

    .subscribe()

//conumser1是活跃消费者， consumer2是备用消费者
```

```
//consumer1收到5条消息后断开连接， consumer2作为活跃消费者接管
```

多个消费者可以附加到同一个订阅上，但只有第一个消费者处于活跃状态，其他消费者处于待机状态。当活跃消费者断开连接时，消息将被派发给其中一个备用消费者，然后备用消费者成为活跃消费者。

如果第一个活跃的消费者在收到5条消息后断开连接，则备用消费者成为活跃的消费者。

consumer1将收到：

```
("key-1", "message-1-1")

("key-1", "message-1-2")

("key-1", "message-1-3")

("key-2", "message-2-1")

("key-2", "message-2-2")
```

consumer2将收到：

```
("key-2", "message-2-3")

("key-3", "message-3-1")

("key-3", "message-3-2")

("key-4", "message-4-1")

("key-4", "message-4-2")
```

注意：如果topic是分区主题，每个分区只有一个活跃消费者，一个分区的消息只分发给一个consumer，多个分区的消息分发给多个consumer。

6.8.3. Shared

创建新的消费者并使用Shared订阅类型订阅。

```
Consumer consumer1 = client.newConsumer()

    .topic("my-topic")

    .subscriptionName("my-subscription")

    .subscriptionType(SubscriptionType.Shared)

    .subscribe()
```

```
Consumer consumer2 = client.newConsumer()

    .topic("my-topic")

    .subscriptionName("my-subscription")

    .subscriptionType(SubscriptionType.Shared)

    .subscribe()

//consumer1和consumer2都是活跃消费者
```

在Shared订阅类型中，多个消费者可以附加到同一个订阅，消息以轮询方式在消费者之间分发。

如果代理一次仅发送一条消息，则：

consumer1会收到以下信息：

```
("key-1", "message-1-1")

("key-1", "message-1-3")

("key-2", "message-2-2")

("key-3", "message-3-1")

("key-4", "message-4-1")
```

consumer2收到以下信息：

```
("key-1", "message-1-2")

("key-2", "message-2-1")

("key-2", "message-2-3")

("key-3", "message-3-2")

("key-4", "message-4-2")
```

Shared订阅不同于Exclusive和Failover订阅类型。Shared订阅具有更好的灵活性，但不能保证顺序性。

6.8.4. Key_shared

创建新的消费者并使用Key_Shared订阅类型订阅。

```

Consumer consumer1 = client.newConsumer()

    .topic("my-topic")

    .subscriptionName("my-subscription")

    .subscriptionType(SubscriptionType.Key_Shared)

    .subscribe()

Consumer consumer2 = client.newConsumer()

    .topic("my-topic")

    .subscriptionName("my-subscription")

    .subscriptionType(SubscriptionType.Key_Shared)

    .subscribe()

///consumer1和consumer2都是活跃消费者
    
```

同Shared订阅一样，Key_Shared订阅类型中的所有消费者都可以附加到同一个订阅。但Key_Shared订阅类型与Shared订阅的不同在于，在Key_Shared订阅类型中，具有相同key的消息按顺序只传递给一个消费者。默认情况下，事先并不知道哪些key会被分配给一个消费者，但一个key只会同时被分配给一个消费者。

consumer1收到以下信息：

```

("key-1", "message-1-1")

("key-1", "message-1-2")

("key-1", "message-1-3")

("key-3", "message-3-1")

("key-3", "message-3-2")
    
```

consumer2 收到以下信息：

```

("key-2", "message-2-1")

("key-2", "message-2-2")

("key-2", "message-2-3")
    
```



```
("key-4", "message-4-1")
```

```
("key-4", "message-4-2")
```

如果在生产者端启用了批处理，则默认情况下会将具有不同key的消息添加到一个批处理中。broker会将批处理派发给消费者，因此默认的批处理机制可能会破坏Key_Shared订阅保证的消息分发语义。生产者需要使用KeyBasedBatcher。

```
Producer producer = client.newProducer()

    .topic("my-topic")

    .batcherBuilder(BatcherBuilder.KEY_BASED)

    .create();
```

或生产者可以禁用批处理。

```
Producer producer = client.newProducer()

    .topic("my-topic")

    .enableBatching(false)

    .create();
```

注意：如果不指定消息键，没有键的消息默认按顺序发送给一个消费者。

6.9. 消息拦截器

ConsumerInterceptors拦截并可能改变消费者收到的消息。

该接口有六个主要事件：

- beforeConsume在receive()或receiveAsync()返回消息之前触发。您可以修改此事件中的消息。
- onAcknowledge在消费者向broker发送确认之前被触发。
- onAcknowledgeCumulative在消费者向broker发送累积确认之前触发。
- onNegativeAcksSend当发生否定确认的重传消息时触发。
- onAckTimeoutSend当发生确认超时的重传时触发。
- onPartitionsChange当（分区的）主题的分区更改时触发。

为了拦截消息，可以在创建Consumer时添加一个或多个ConsumerInterceptor，如下所示：

```
Consumer<String> consumer = client.newConsumer()
```

```
.topic("my-topic")

.subscriptionName("my-subscription")

.intercept(_new_ ConsumerInterceptor<String> {

    @Override

    public Message<String> beforeConsume(Consumer<String> consumer, Message<String> message) {

        //用户自定义处理逻辑

    }

    @Override

    public void onAcknowledge(Consumer<String> consumer, MessageId messageId, Throwable cause) {

        //用户自定义处理逻辑

    }

    @Override

    public void onAcknowledgeCumulative(Consumer<String> consumer, MessageId messageId, Throwable cause) {

        //用户自定义处理逻辑

    }

    @Override

    public void onNegativeAcksSend(Consumer<String> consumer, Set<MessageId> messageIds) {

        //用户自定义处理逻辑

    }

    @Override

    public void onAckTimeoutSend(Consumer<String> consumer, Set<MessageId> messageIds) {

        //用户自定义处理逻辑

    }

}
```

```
    }

    @Override

    public void onPartitionsChange(String topicName, _int_ partitions) {

        //用户自定义处理逻辑

    }

})

.subscribe();
```

注意：如果使用多个拦截器，它们将按照传递给intercept方法的顺序应用。

7. Reader

通过reader接口，客户端可以在主题中“手动定位”，并读取指定消息之后的所有消息。Java API使用户能够通过指定主题和MessageId创建Reader对象。

示例如下：

```
byte[] msgIdBytes = // Some message ID byte array

MessageId id = MessageId.fromByteArray(msgIdBytes);

Reader reader = pulsarClient.newReader()

    .topic(topic)

    .startMessageId(id)

    .create();

while (true) {

    Message message = reader.readNext();

    //进程信息

}
```

在以上示例中，一个特定的主题和消息（通过消息ID）被实例化为一个Reader对象；在消息被msgIdBytes标识之后，Reader遍历主题中的每个消息（如何获得该值取决于应用程序）。

以上示例显示将Reader对象指向特定消息（通过ID），但也可以使用MessageId.earliest来指向主题上最早的可用消息，或使用MessageId.latest来指向最新的可用消息。

7.1. 配置Reader

创建reader时，可以使用loadConf配置。在loadConf中可使用以下参数配置。

参数名	类型	描述	默认值
topicName	String	主题名称。	-

参数名	类型	描述	默认值
receiverQueueSize	int	消费者接收队列的大小。 例如，在应用程序调用Receive之前，消费者可以累积的消息数。 高于默认值会增加消费者吞吐量，但会牺牲更多内存利用率。	1000
readerListener	ReaderListener<T>	接收到的消息时调用的监听器。	-
readerName	String	Reader名称。	-
subscriptionName	String	订阅名称	单个主题时，默认订阅名称为"reader-" + 10位数的UUID； 当有多个主题时，默认订阅名称为"multiTopicsReader-" + 10位数的UUID。
subscriptionRolePrefix	String	订阅角色的前缀。	-
cryptoKeyReader	CryptoKeyReader	抽象出对密钥存储的访问接口。	-
cryptoFailureAction	ConsumerCryptoFailureAction	消费者在收到无法解密的消息时应采取措施。 • FAIL：默认值，使消息失败，直到加密成功。 • DISCARD：静默确认并且不向应用程序传递消息。 • CONSUME：将加密消息传递给应用程序，解密消息由应用程序负责。 消息解压失败。 如果消息包含批处理消息，则客户端无法批量检索单个消息。 传递的加密消息包含{@link EncryptionContext}，其中包含加密和压缩信息，应用程序可以使用这些信息解密消耗的消息有效负载。	ConsumerCryptoFailureAction.FAIL

参数名	类型	描述	默认值
readCompacted	boolean	如果启用readCompacted，消费者将从压缩主题中读取消息，而不是主题的完整消息积压。 消费者只能看到压缩主题中每个键的最新值，直到达到主题消息中压缩积压的点。超过这个点，照常发送消息。 readCompacted只能在具有单个活跃消费者（例如，灾备(Failover)或独占(Exclusive)订阅）的持久性主题订阅上启用。 尝试在订阅非持久主题或共享订阅时启用，会导致订阅调用抛出PulsarClientException。	False
resetIncludeHead	boolean	如果设置为true，则返回的第一条消息是由messageId指定的消息。 如果设置为false，则返回的第一条消息是messageId指定的消息之后的消息。	False

7.2. Sticky key范围Reader

在Sticky key范围Reader中，broker只会发送消息key的哈希值包含在指定的key哈希值范围内的消息。可以在Reader上指定多个key哈希值范围。

以下是创建Sticky key范围Reader的示例：

```
pulsarClient.newReader()

    .topic(topic)

    .startMessageId(MessageId.earliest)

    .keyHashRange(Range.of(0, 10000), Range.of(20001, 30000))

    .create();
```

总哈希范围大小为65536，所以范围的最大端应小于等于65535。

8. Schema

在TongLINK/Q-CN中，所有消息数据都是由字节数组组成。消息Schema使用户能够在构造和处理消息时使用其他类型的数据（从字符串等简单类型到更复杂的特定应用类型）。例如，如果在不指定模式的情况下构建一个生产者，那么该生产者只能生产byte[]类型的消息。示例如下：

```
Producer<_byte_[]> producer = client.newProducer()

    .topic(topic)

    .create();
```

以上示例中的生产者相当于一个Producer<byte[]>（实际使用中应该总是明确指定类型）。如果想对不同类型的数据使用生产者，则需要指定一个schema来告知TongLINK/Q-CN哪种数据类型将通过主题传输。

8.1. AvroBaseStructSchema示例

假设有一个SensorReading类要通过主题传输：

```
public class SensorReading {

    public float temperature;

    public SensorReading(float temperature) {

        this.temperature = temperature;

    }

    //需要一个无参数的构造器

    public SensorReading() {

    }

    public float getTemperature() {

        return temperature;

    }

    public void setTemperature(_float_ temperature) {
```

```

        this.temperature = temperature;
    }
}

```

然后可以创建一个Producer<SensorReading>（或Consumer<SensorReading>）：

```

Producer<SensorReading> producer = client.newProducer(JSONSchema.of(SensorReading._class_))

    .topic("sensor-readings")

    .create();

```

以下是目前可用于Java的schema：

- 无schema或字节数组schema（可以使用Schema.BYTES）：

```

Producer<byte[]> bytesProducer = client.newProducer(Schema.BYTES)

    .topic("some-raw-bytes-topic")

    .create();

```

或者等效于：

```

Producer<byte[]> bytesProducer = client.newProducer()

    .topic("some-raw-bytes-topic")

    .create();

```

- 对于UTF-8编码的字符串数据，使用Schema.STRING：

```

Producer<String> stringProducer = client.newProducer(Schema.STRING)

    .topic("some-string-topic")

    .create();

```

- 使用Schema.JSON为POJO创建JSON schema：

```

Producer<MyPojo> pojoProducer = client.newProducer(Schema.JSON(MyPojo.class))

    .topic("some-pojo-topic")

    .create();

```


- 使用Schema.PROTOBUF生成Protobuf schema。以下示例展示如何创建Protobuf schema并使用它来实例化一个新的生产者：

```
Producer<MyProtobuf> protobufProducer = client.newProducer(Schema.PROTOBUF(MyProtobuf.class))

    .topic("some-protobuf-topic")

    .create();
```

- 使用 Schema.AVRO定义Avro schema。以下代码片段演示如何创建和使用Avro schema：

```
Producer<MyAvro> avroProducer = client.newProducer(Schema.AVRO(MyAvro.class))

    .topic("some-avro-topic")

    .create();
```

9. 认证

TongLINK/Q-CN目前支持的身份验证方式为：JWT。可以在Java客户端中使用这种认证方式。

9.1. JWT认证

要使用JWT认证，需要管理控制台的“租户管理员”创建客户端token并给相应的命名空间或者主题授权。“租户管理员”提供对应的客户端token给客户端，客户端使用token进行收发消息的操作。

示例如下：

```
PulsarClient client = PulsarClient.builder()

    .serviceUrl("pulsar://broker.example.com:6650/")

    .authentication(

AuthenticationFactory.token("eyJhbGciOiJIUzI1NiJ9.eyJzdWUiOiJKb2UifQ.ipevRNuRP6HflG8cFKnmUPtypruRC4fb1DWtoLL62SY"))

    .build();
```

10. 国密

作为业务的中央消息总线，TongLINK/Q-CN经常用于存储关键任务的数据。因此，在TongLINK/Q-CN中启用安全功能至关重要。本章介绍了TongLINK/Q-CN用于保护数据的主要安全控制。

使用国密加解密，客户端需要先依赖国密SDK，当前仅支持jdk1.8 8u161之后至java 11的版本。

依赖国密SDK方法如下：

1. 获取安装包tlq-cn-client-gm.zip；
2. 解压安装包；
3. 将sdk安装至maven仓库：
 - Windows平台执行**installMaven.bat**脚本把sdk安装至maven仓库
 - Linux平台执行**installMaven.sh**脚本把sdk安装至maven仓库
4. 在客户端pom.xml文件的<dependencies>中添加以下内容：

```
<dependency>

    <groupId>com.tongtech.client</groupId>

    <artifactId>tlq-cn-gm</artifactId>

    <version>10.0.x.x</version>

</dependency>

<dependency>

    <groupId>com.tencent.kona</groupId>

    <artifactId>kona-crypto</artifactId>

    <version>1.0.9</version>

</dependency>

<dependency>

    <groupId>com.tencent.kona</groupId>

    <artifactId>kona-pkix</artifactId>

    <version>1.0.9</version>
```

```

</dependency>

<dependency>

    <groupId>com.tencent.kona</groupId>

    <artifactId>kona-ssl</artifactId>

    <version>1.0.9</version>

</dependency>

<dependency>

    <groupId>com.tencent.kona</groupId>

    <artifactId>kona-provider</artifactId>

    <version>1.0.9</version>

</dependency>

```

说明：在实际使用中需将<version>修改为TongLINK/Q-CN的具体版本号。

10.1. 端到端加密

应用程序可以使用TongLINK/Q-CN端到端加密（E2EE）在生产者端对消息进行加密，在消费者端对消息进行解密。用户可以使用应用程序配置的公钥和私钥对来执行加密和解密操作。只有拥有有效密钥的消费者才能解密加密的消息。

生成公私钥方法如下：

客户端引入国密依赖后，使用工具类**GeneratePemTool**生成公钥和私钥文件。

- 生成公钥和私钥到指定路径：

```
GeneratePemTool.generateKeyPemFile("path");
```

路径参数path不能为空。

- 生成公钥和私钥到当前路径：

```
GeneratePemTool.generateKeyPemFile();
```

客户端使用示例如下：

```
PulsarClient pulsarClient = PulsarClient.builder().serviceUrl("pulsar://localhost:6650").build();
```

```
String topic = "persistent://my-tenant/my-ns/my-topic";

CryptoKeyReader keyReader = new RawFileKeyReader("test_ecdsa_pubkey.pem", "test_ecdsa_privkey.pem");

Producer<byte[]> producer = pulsarClient.newProducer()

    .topic(topic)

    .cryptoKeyReader(keyReader)

    .addEncryptionKey("myappkey")

    .messageCrypto(new MessageCryptoGm(true)) // 不使用国密无需配置这一步

    .create();

Consumer<byte[]> consumer = pulsarClient.newConsumer()

    .topic(topic)

    .subscriptionName("my-subscriber-name")

    .cryptoKeyReader(keyReader)

    .messageCrypto(new MessageCryptoGm(false)) // 不使用国密无需配置这一步

    .subscribe();

Reader<byte[]> reader = pulsarClient.newReader()

    .topic(topic)

    .startMessageId(MessageId.earliest)

    .cryptoKeyReader(keyReader)

    .messageCrypto(new MessageCryptoGm(false)) // 不使用国密无需配置这一步

    .create();

编写密钥读取类CryptoKeyReader:

class RawFileKeyReader implements CryptoKeyReader {
```

```
String publicKeyFile = "";

String privateKeyFile = "";

RawFileKeyReader(String pubKeyFile, String privKeyFile) {

    publicKeyFile = pubKeyFile;

    privateKeyFile = privKeyFile;

}

@Override

public EncryptionKeyInfo getPublicKey(String keyName, Map<String, String> keyMeta) {

    EncryptionKeyInfo keyInfo = new EncryptionKeyInfo();

    try {

        keyInfo.setKey(Files.readAllBytes(Paths.get(publicKeyFile)));

    } catch (IOException e) {

        System.out.println("ERROR: Failed to read public key from file " + publicKeyFile);

        e.printStackTrace();

    }

    return keyInfo;

}

@Override

public EncryptionKeyInfo getPrivateKey(String keyName, Map<String, String> keyMeta) {

    EncryptionKeyInfo keyInfo = new EncryptionKeyInfo();

    try {

        keyInfo.setKey(Files.readAllBytes(Paths.get(privateKeyFile)));

    } catch (IOException e) {

        System.out.println("ERROR: Failed to read private key from file " + privateKeyFile);

        e.printStackTrace();

    }

    return keyInfo;

}
```

```

    } catch (IOException e) {

        System.out.println("ERROR: Failed to read private key from file " + privateKeyFile);

        e.printStackTrace();

    }

    return keyInfo;

}

}

```

10.2. 国密TLS

应用程序可以使用TongLINK/Q-CN国密TLS通信协议来保证通信过程中的数据安全。基于国密双证书双向认证保证来保证通信层的数据安全。

客户端使用示例如下：

```

PulsarClient client = PulsarClient.builder()

    // 配置gmtls通信地址

    .serviceUrl("pulsar+ssl://host:port")

    .sslFactoryPlugin(GmPulsarSslFactory.class.getName())

    .sslFactoryPluginParams("gmCaCertFilePath:E:/gm/chain-ca.crt,"

        + "gmSignCertFilePath:E:/gm/client_sign.crt,gmSignKeyFilePath:E:/gm/client_sign.key,"

        + "gmEncCertFilePath:E:/gm/client_enc.crt,gmEncKeyFilePath:E:/gm/client_enc.key,"

        + "gmKeyPassword:ATE9XA+8p1hX5pZfovujRA==")

    .build();

```

11. 消息过滤

消费者订阅了某个主题后，TongLINK/Q-CN会将该主题中的所有消息投递给消费者。若消费者只需要关注部分消息，可通过设置过滤条件在服务端进行过滤，只获取到需要关注的消息，避免接收到大量无效的消息。以此，也可以简化业务逻辑的架构设计。客户端要使用此功能时，需先确认服务端已经开启了此特性。

TongLINK/Q-CN支持两种类型的过滤方式：

- 标签过滤，生产消息时打上一个或多个固定Tag，消费者通过指定Tag订阅消息。
- SQL过滤，生产消息时打上一个或多个k-v的属性，消费者可以通过更为灵活的SQL 92语法来订阅消息。

两种过滤方式的对比如下：

对比项	标签过滤	SQL过滤
过滤目标	消息的Tag标签属性	消息的k-v属性
过滤能力	正则表达式匹配	SQL语法匹配
适用场景	简单过滤场景、计算逻辑简单轻量	复杂过滤场景、计算逻辑较复杂

11.1. 标签过滤

Tag消息不支持Batch功能，Batch功能默认是开启的。如果要使用Tag消息，需要在Producer侧禁用掉batch，具体如下：

```
// 构建生产者

Producer<byte[]> producer = pulsarClient.newProducer()

// 禁用掉batch功能

.enableBatching(false)

// topic完整路径，格式为persistent://集群（租户）ID/命名空间/Topic名称

.topic("persistent://pulsar-xxx/sdk_java/topic2").create();
```

- Tag消息的过滤只针对已设置Tag的消息，消费者订阅Topic时若未设置Tag，Topic中的所有消息都将被投递到消费端进行消费。
- 如果要开启Tag消息，需要发送消息的时候，在ProducerMessage中设置Properties字段；同时在创建Consumer的时候需要在ConsumerOptions中指定SubscriptionProperties字段。
- 在ProducerMessage中设置Properties字段时，其中key为tag的名字，value为固定值：TAGS。
- 在ConsumerOptions中指定SubscriptionProperties字段时，其中key为要订阅的tag的名字，value为tag的版本信息，为保留字段，目前没有实质含义，用来做后续功能的扩展，具体如下：

11.1.1. 指定单个tag

```
// 发送消息

MessageId msgId = producer.newMessage()

    .property("tag1", "TAGS")

    .value(value.getBytes(StandardCharsets.UTF_8))

    .send();

// 订阅相关参数， 可用来设置订阅标签(TAG)

HashMap<String, String> subProperties = new HashMap<>();

subProperties.put("tag1","1");

// 构建消费者

Consumer<byte[]> consumer = pulsarClient.newConsumer()

// topic完整路径， 格式为persistent://集群（租户）ID/命名空间/Topic名称， 从【Topic管理】处复制

    .topic("persistent://pulsar-xxxx/sdk_java/topic2")

// 需要在控制台Topic详情页创建好一个订阅， 此处填写订阅名

    .subscriptionName("topic_sub1")

// 声明消费模式为共享模式

    .subscriptionType(SubscriptionType.Shared)

// 订阅相关参数， tag订阅等。。

    .subscriptionProperties(subProperties)

// 配置从最早开始消费， 否则可能会消费不到历史消息

    .subscriptionInitialPosition(SubscriptionInitialPosition.Earliest).subscribe();
```

11.1.2. 指定多个tag

```
// 发送消息

MessageId msgId = producer.newMessage()

    .property("tag1", "TAGS")

    .property("tag2", "TAGS")

    .value(value.getBytes(StandardCharsets.UTF_8))

    .send();

// 订阅相关参数， 可用来设置订阅标签(TAG)

HashMap<String, String> subProperties = new HashMap<>();

subProperties.put("tag1","1");

subProperties.put("tag2","1");

// 构建消费者

Consumer<byte[]> consumer = pulsarClient.newConsumer()

// topic完整路径， 格式为persistent://集群（租户）ID/命名空间/Topic名称， 从【Topic管理】处复制

    .topic("persistent://pulsar-xxxx/sdk_java/topic2")

// 需要在控制台Topic详情页创建好一个订阅， 此处填写订阅名

    .subscriptionName("topic_sub1")

// 声明消费模式为共享模式

    .subscriptionType(SubscriptionType.Shared)

// 订阅相关参数， tag订阅等。。

    .subscriptionProperties(subProperties)

// 配置从最早开始消费， 否则可能会消费不到历史消息

    .subscriptionInitialPosition(SubscriptionInitialPosition.Earliest).subscribe();
```

注意：

1. 单个消费者可以使用多个Tag，多个Tag之间的关系是「或」。
2. 多个消费者需要使用相同Tag。若一个订阅中有不同消费者使用不同Tag，则会出现过滤规则覆盖的情况，进而影响业务逻辑。

11.2. SQL过滤

11.2.1. 生产者示例

生产者可以在property中添加多个属性。

```
// 构建生产者

Producer<byte[]> producer = pulsarClient.newProducer()

// 禁用掉batch功能

.enableBatching(false)

// topic完整路径，格式为persistent://集群（租户）ID/命名空间/Topic名称

.topic("persistent://pulsar-xxx/sdk_java/topic2").create();

// 发送消息

MessageId msgId = producer.newMessage()

.property("idc", "idc1") // 指定消息的属性（idc）

.property("label", "online") // 指定消息的属性（label）

.property("other", "xxx") // 指定消息的其他属性

.value(value.getBytes(StandardCharsets.UTF_8))

.send();
```

11.2.2. 消费者示例

消费者的properties中必须包含**TLQ_CN_SQL92_FILTER_EXPRESSION**，认为开启了SQL92过滤，value即为SQL 92过滤表达式。

```
// 订阅相关参数
```

```

HashMap<String, String> subProperties = new HashMap<>();

//
消费者的properties中包含<strong>TLQ_CN_SQL92_FILTER_EXPRESSION</strong>, 认为开启了SQL92过滤, valu
e即为过滤表达式。

subProperties.put("<strong>TLQ_CN_SQL92_FILTER_EXPRESSION</strong> ", "idc = 'idc1' AND label IS NOT
NULL"); // 表达式说明: idc属性等于idc1 并且 label属性存在

// 构建消费者

Consumer<byte[]> consumer = pulsarClient.newConsumer()

    // topic完整路径, 格式为persistent://集群 (租户) ID/命名空间/Topic名称, 从【Topic管理】处复制

    .topic("persistent://pulsar-xxxx/sdk_java/topic2")

    // 需要在控制台Topic详情页创建好一个订阅, 此处填写订阅名

    .subscriptionName("topic_sub1")

    // 声明消费模式为共享模式

    .subscriptionType(SubscriptionType.Shared)

    // 订阅配置相关参数, 里面携带SQL过滤表达式

    .subscriptionProperties(subProperties)

    .subscribe();

```

注意事项

1. 如果subscriptionProperties中包含了**TLQ_CN_SQL92_FILTER_EXPRESSION**, 即认为此订阅使用SQL过滤。如果同时subscriptionProperties中还存在其他properties, 其他properties忽略, 即不再使用标签过滤。
2. SQL过滤是根据消息中的属性属性 (即发送消息时候指定的property) 进行过滤的, 和标签 (Tag) 过滤是相互独立的。即一旦使用了SQL过滤, 完全按照消费属性来过滤, 不再区分是否是TAGS标签的属性。对于property中key=tag1, value=TAGS的属性, Pulsar将认为这是一个普通的property, key=tag1, value=TAGS字符串的属性, 不会特殊对待。
3. 消息过滤不支持Batch功能, Batch功能默认是开启的, 需要在创建Producer的时候禁用掉batch。如果是批量的消息, 服务端不会执行消息过滤, 会直接投递给消费者。

4. SQL语句中支持的属性（property）的命名格式只能由下面的字符组成：字母、数字、下划线。
5. SQL语句中的判断条件最大不能超过50个，建议不超过5个。换句话说，就是SQL表达式语句中的AND和OR的数量要小于50个。**BETWEEN xxx AND xxx**和**NOT BETWEEN xxx AND xxx**中包含的AND也会计算在内。
6. 如果新传入的SQL表达式规则不正确，导致服务端无法正确解析对应的SQL语句，则服务端会继续保持和之前一致的过滤方式。
7. 由于SQL属性过滤是生产者定义消息属性，消费者设置SQL过滤条件，因此过滤条件的计算结果具有不确定性，服务端的处理方式如下：
 - 异常情况处理：如果过滤条件的表达式计算抛异常，消息默认被过滤，不会被投递给消费者。例如比较数字和非数字类型的值。
 - 空值情况处理：如果过滤条件的表达式计算值为null或不是布尔类型（true和false），则消息默认被过滤，不会被投递给消费者。例如发送消息时未定义某个属性，在订阅时过滤条件中直接使用该属性，则过滤条件的表达式计算结果为null。
 - 数值类型不符处理：如果消息自定义属性为浮点型，但过滤条件中使用整数进行判断，则消息默认被过滤，不会被投递给消费者。
8. 多个消费者需要使用相同的过滤规则。若一个订阅中有不同消费者使用不同规则，则会出现过滤规则覆盖的情况，进而影响业务逻辑。

11.2.3. SQL过滤语法规则

语法	说明	示例
IS NULL	判断属性不存在。	a IS NULL：属性a不存在。
IS NOT NULL	判断属性存在。	a IS NOT NULL：属性a存在。
> >= < <=	用于比较数字，不能用于比较字符串，否则消费者客户端启动时会报错。 说明： 可转化为数字的字符串也被认为是数字。	a IS NOT NULL AND a > 100：属性a存在且属性a的值大于100。 a IS NOT NULL AND a > 'abc'：错误示例，abc为字符串，不能用于比较大小。
BETWEEN xxx AND xxx	用于比较数字，不能用于比较字符串，否则消费者客户端启动时会报错。等价于>= xxx AND <= xxx。表示属性值在两个数字之间。	a IS NOT NULL AND (a BETWEEN 10 AND 100)：属性a存在且属性a的值大于等于10且小于等于100。
NOT BETWEEN xxx AND xxx	用于比较数字，不能用于比较字符串，否则消费者客户端启动会报错。等价于< xxx OR > xxx，表示属性值在两个值的区间之外。	a IS NOT NULL AND (a NOT BETWEEN 10 AND 100)：属性a存在且属性a的值小于10或大于100。

语法	说明	示例
IN (xxx, xxx)	表示属性的值在某个集合内。集合的元素只能是字符串。	a IS NOT NULL AND (a IN ('abc', 'def')): 属性a存在且属性a的值为abc或def。
= <>	等于和不等于。可用于比较数字和字符串。	a IS NOT NULL AND (a = 'abc' OR a<>'def'): 属性a存在且属性a的值为abc或a的值不为def。
AND OR	逻辑与、逻辑或。可用于组合任意简单的逻辑判断，需要将每个逻辑判断内容放入括号内。	a IS NOT NULL AND (a > 100) OR (b IS NULL): 属性a存在且属性a的值大于100或属性b不存在。

附录 A: 术语说明

名称	说明
TongLINK/Q-CN (简称TLQ-CN)	东方通云原生消息中间件软件
Broker	服务端工作节点
Client	客户端
Topic	主题
Message	消息
Producer	生产者，即消息的提供者
Consumer	消费者，即消息的消费者
Pub-Sub	发布及订阅
Point-to-Point(PTP)	点对点模式，即生产者和消费者之间的消息往来

